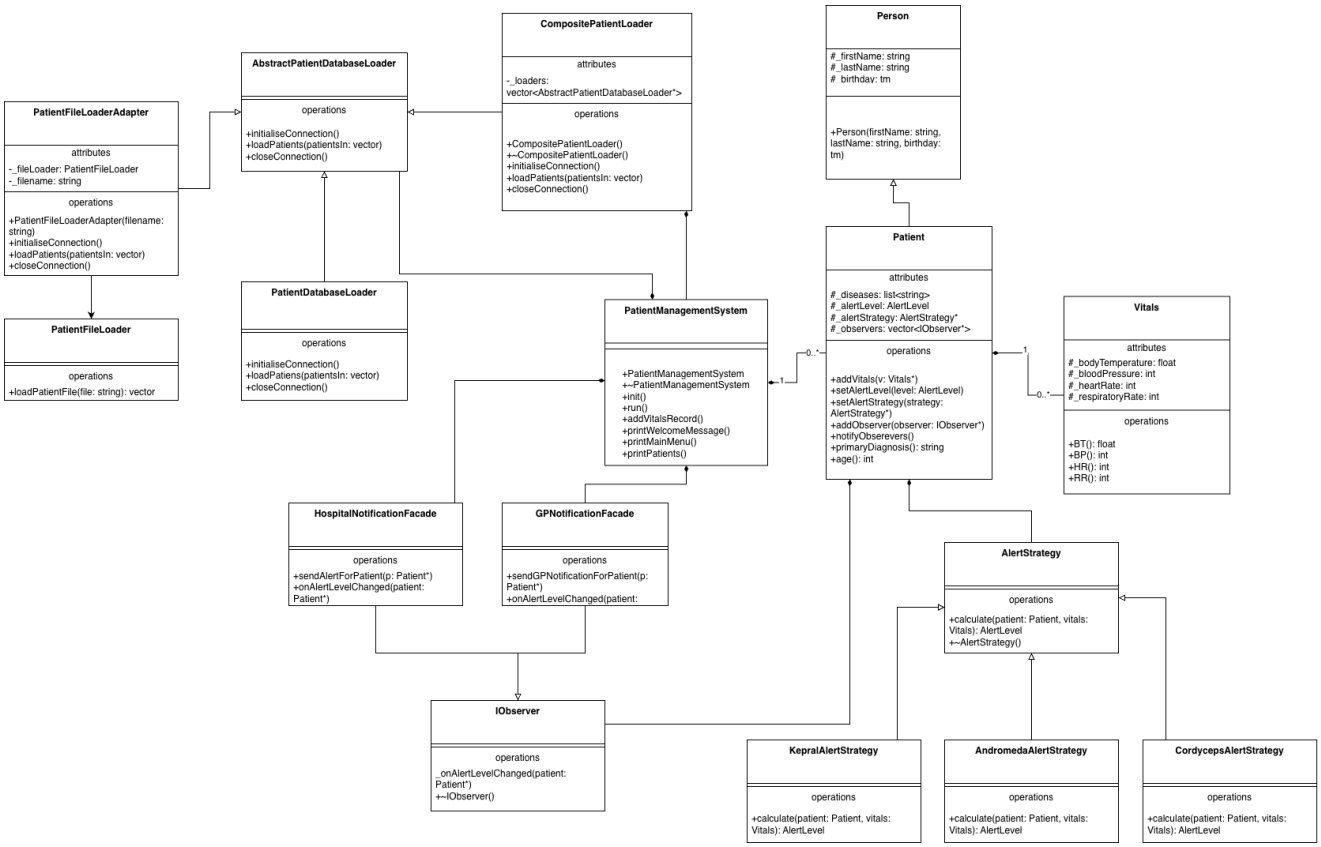
Name: Gia Bach Pham

Student ID: 3160100

Programming Assignment 2: Design Patterns with C++

Overall class diagram



FR1 section — Adapter pattern – load patients from file

The current system requires loading patients from a text file, however the issue is the existing **PatientFileLoader** class had a completely different interface to the **AbstractPatientDatabaseLoader** that **PatientManagementSystem** depends on. Rather than modifying either class, I applied the Adapter pattern which will wrap an incompatible interface inside a new class that matches the expected interface. This means that the system will remain unchanged and will support a new data source which makes switching between loaders a single line of code change in the constructor.

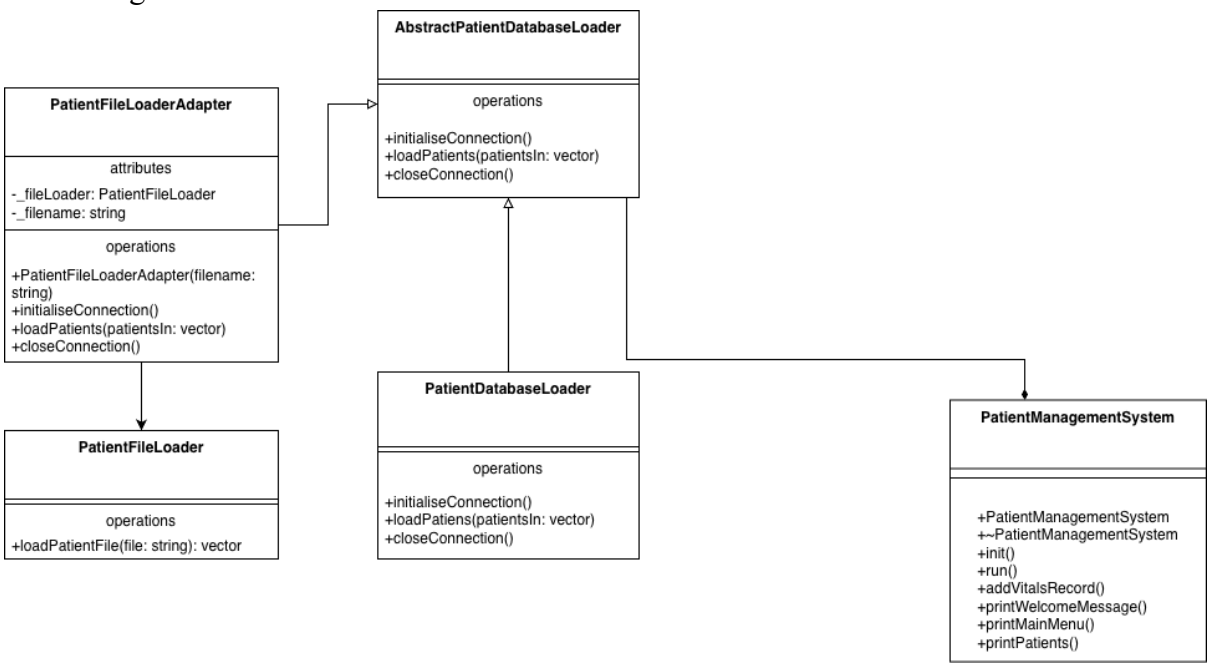
How it works:

1. PatientManagementSystem holds an AbstractPatientDatabaseLoader pointer and calls initialiseConnection(), loadPatients(), and closeConnection() on it.
2. PatientFileLoaderAdapter inherits from AbstractPatientDatabaseLoader so the system accept it without any changes
3. Internally, PatientFileLoaderAdapter contains a PatientFileLoader instance and a filename string
4. When loadPatients() is called, the adapter delegates to PatientFileLoader::loadPatientFile() which parses the text file line by line
5. Each line will be split by | delimiter to extract uid, name, birthday, disease and vital
6. The results are copied into the vector and passed by the system
7. intialiseConnection() and closeConnection() do nothing as no connection is needed for file loading
8. Switching to file loading requires only changing one line in PatientManagement System constructor

Git commits:

- ca528fd – create PatientFileLoadeAdapter header file
- d4c2949 – add PatientFileLoader Adapter header and cpp
- 70855ec – Implement FileLoaderAdapter and partial loadfile method
- e8cde6c – partial implement fileloader method in PatientFileLoader.cpp
- F72ec70 – switch to PatientFileLoaderAdapter

UML Diagram:



FR2 section — Composite pattern – load patients from file and database

The system needs to load patients from both the database and the text file simultaneously, while still keeping the ability to switch between loaders with a single line of code change. I applied the Composite pattern which allows a group of objects to be treated as a single object. The **CompositePatientLoader** will contain multiple loaders and calls each one in sequence, which makes it transparent to PatientManagementSystem which has no knowledge of how many loaders are running underneath.

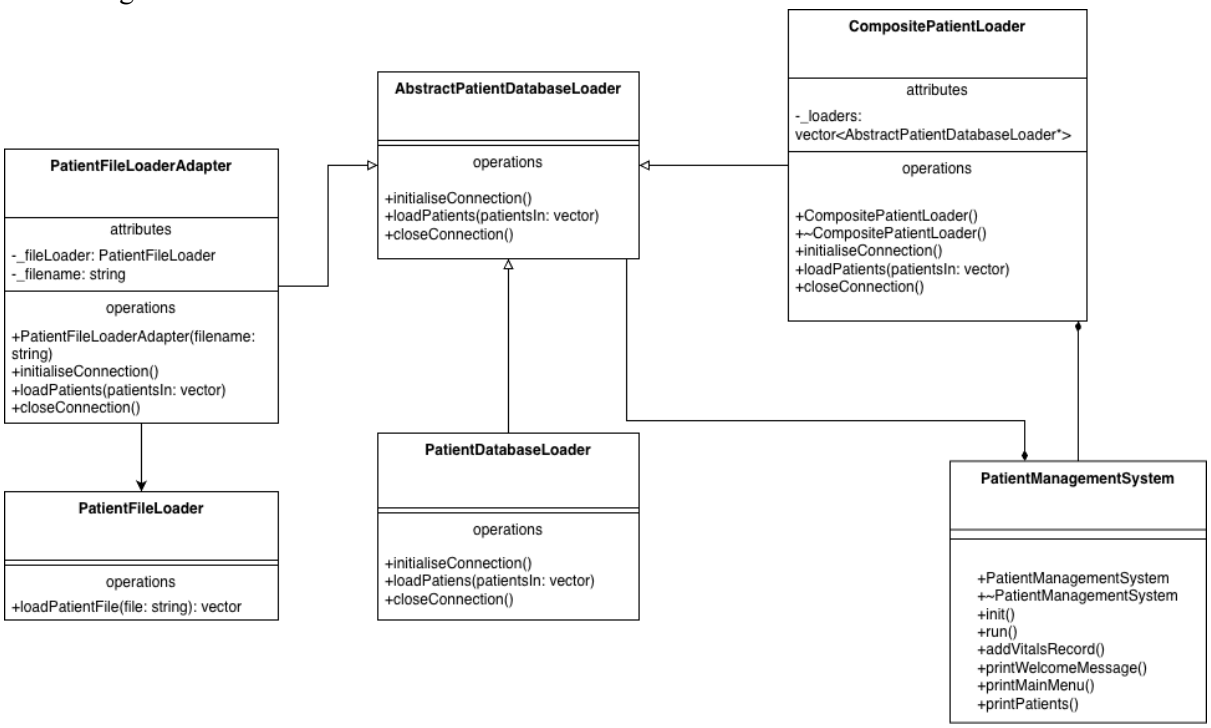
How it works:

1. CompositePatientLoader inherits from AbstractPatientDatabaseLoader so the system accept it as a single loader
2. In its constructor, it creates and stores both PatientDatabaseLoader and a PatientFileLoaderAdapter in a vector of AbstractPatientDatabaseLoader*
3. When intialiseConnection() is called, it loops through all loaders and call initialiseConnection() on each
4. When loadPatients() is called, it loops through all loader and calls loadPatients() on each, appending results to that specific vector
5. When closeConnection() is called, it loops through all loaders and closes each connection
6. Switching to composite loading requires only changing one line in PatientManagementSystem
7. The destructor cleans up all loader memory to prevent leaks

Git commits:

- 3e5eb6f – create an implement CompositePatientLoader file
- 4b53284 – implement CompositePatientLoader.cpp
- a59c501 – implement Composite pattern to load patients from file and database

UML Diagram:



FR3 section — Strategy pattern – calculate patients alert level

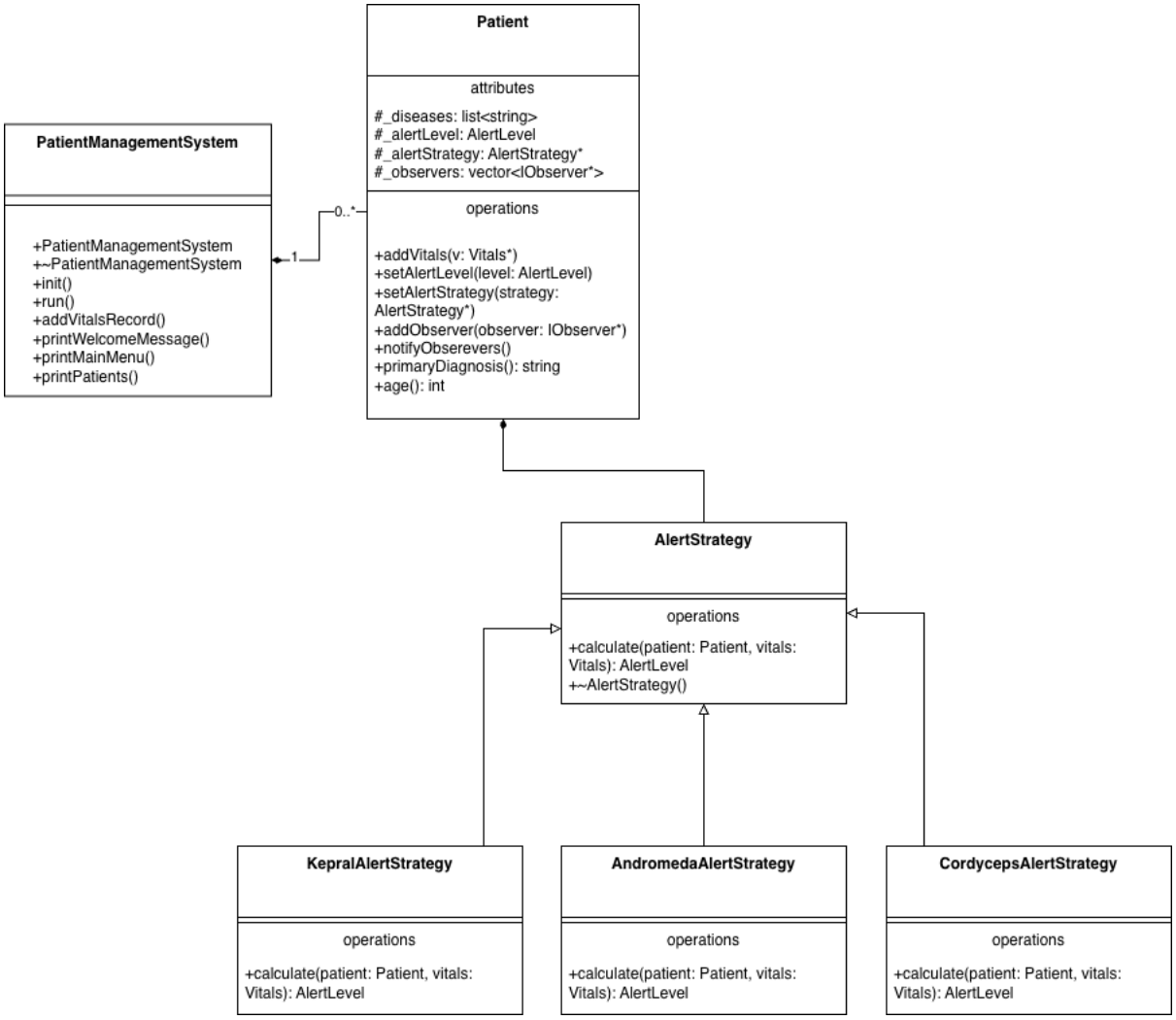
Each patient's disease will require the system to calculate alert levels differently. If place all disease logic inside the **Patient** class would violate the open/closed principle, which means every new disease would require modifying Patient. I applied the Strategy Pattern which encapsulate each algorithm in its own class and makes them interchangeable. The Patient class simply calls calculate() on its assigned strategy without knowing which one it is, keeping the alert logic separate from the patient data.

1. AlertStrategy is an abstract base class with pure virtual calculate() method that returns an AlertLevel
2. Three concrete strategies created – CordycepsAlertStrategy, KepralAlertStrategy, and AndromedaAlertStrategy – each implementing the alert rules from Table 1.
3. Patients holds an AlertStrategy* pointer initialised to nullptr
4. In PatientManagementSystem::init(), after all patients are loaded, each patients will be assigned the correct strategy based on their priary diagnosis using setAlertStrategy()
5. When addVitals() is called interactively, it calls _alertStrategy->calculate(*this, *v) which returns the calculated alert level
6. The patient then calls setAlertLevel() on itself with the return level
7. Historical vitals loaded from file or database do not trigger calculations because no strategy is assigned at load time
8. The patient destructor cleans up the strategy memory

Git commit:

- 1635bc4 — create AlertStrategy header file
- 136f95f — implement CordycepsAlertStrategy header
- 246adc6 — implement CordycepsAlertStrategy cpp
- b0fd321 — implement Andromeda and Kepral header files
- 2156160 — implement Kepral and Andromeda cpp logic
- 10804c7 — add alert strategy support to Patient class
- 95aa8cb — add alert strategy to patient on initialisation
- a02e8a0 — add Kepral and Andromeda diagnosis for patient management system initialisation

UML Diagram:



FR4 section — Observer pattern – alert Hospitals and GPs

The system needs to automatically notify hospitals and GPs when a patient's alert level reached Red. Hardcoding the notification calls would create tight coupling between the patient and notification systems. I applied the Observer pattern which allows objects to automatically notify a list of dependents when their state changes, without knowing who those dependants are. This means new notification system can be added in the future without modifying Patient.

1. IObserver is an abstract interface with a pure virtual onAlertLevelChanged(Patient* patient) method
2. HospitalAlertSystemFacade and GPNotificationSystemFacade both inherit from IObserver and implement onAlertLevelChanged(), which delegates to their existing alert methods
3. Patient holds a vector of IObserver* and provides addObserver() and notifyObserver() methods

4. In PatientManagementSystem::init(), both facades are registetered as observers on every patients using addObserver()
5. When setAlertLevel() is called with AlertLevel::Red, Patient calls notifyObservers()
6. NotifyObservers() loops through all registered observers and calls onAlertLevelChanged(this) on each
7. The hosptial facade prints a critical alert messsage, the GP facade prints a follow-up notification
8. Notification happens immediately and automatically when a Red alert is triggered

Git commits:

- cbdafdb — create IObserver.h
- e28c170 — call notifyObservers in setAlertLevel
- a3b79b4 — implement Observer pattern and fix bugs

UML Diagram: